Locality-First Messaging: Decoupling Availability and Consensus

Narayan Krishna R
nrajeshk@usc.edu

Kriti Bhattacharya
kritibha@usc.edu

VenkataramanVenkatasubaniam
vv70613@usc.edu

Abstract

We present a distributed log replication system that combines gossip-based synchronization with deterministic ordering and control-plane authorization. Nodes exchange summaries to efficiently converge without redundant data transfer, while Lamport clocks ensure consistent ordering across replicas. An epoch-based mechanism enforces membership by rejecting invalid updates before merge. Our evaluation shows fast convergence (~100 ms), no duplication under replay, and correct enforcement of access constraints.

1. Introduction

Distributed systems are widely used to maintain replicated state across multiple nodes, but ensuring that all nodes converge to a consistent view remains a fundamental challenge. This is particularly difficult under asynchronous communication, where messages may be delayed, reordered, or duplicated. For example, if one node appends an entry while another is offline, the system must later reconcile their states without introducing inconsistencies or duplicating updates.

In this project, we design a distributed log replication system that addresses these challenges through gossip-based synchronization and deterministic ordering. Nodes exchange compact summaries of their state and only transfer log entries when necessary, reducing redundant communication while ensuring eventual convergence. Ordering is enforced using Lamport clocks with deterministic tie-breaking, allowing all nodes to resolve concurrent updates consistently.

We further integrate an epoch-based authorization mechanism, where a control-plane service validates each message before it is merged. This prevents stale or unauthorized updates from entering the system while preserving convergence among valid entries. Our evaluation shows that the system achieves fast convergence, avoids duplication under repeated synchronization, and enforces correctness constraints effectively.

2. System Design

2.1 Overview

The system is designed around a key architectural principle: decoupling availability from consistency. Instead of enforcing global ordering at the time of message ingestion, nodes are allowed to accept local writes immediately and reconcile state asynchronously. This design enables the system to remain responsive under network delays, partitions, and intermittent connectivity, while still ensuring eventual consistency.

At a high level, the system consists of a set of peer nodes that maintain replicated append-only logs, a lightweight client that initiates updates, and a control-plane service responsible for enforcing membership rules. This approach follows a locality-first model, where operations are applied locally and propagated later, rather than relying on a centralized coordinator or strict global ordering. As motivated in our proposal, this design is particularly useful in scenarios such as offline messaging or partitioned networks, where immediate availability is more critical than strict consistency .

2.2 Data Plane: Gossip-Based Synchronization

The data plane is responsible for maintaining and synchronizing replicated logs across nodes. Each node stores messages in an append-only log and exchanges state with peers using a gossip protocol.

Instead of transmitting full logs, nodes first exchange compact summaries, which include metadata such as the number of entries and the maximum logical timestamp. These summaries allow nodes to determine whether their states differ. If the summaries match, no further action is taken; otherwise, the node sends the missing log entries.

This mechanism ensures convergence while minimizing redundant communication and synchronization overhead.

A key property of this approach is that it is inherently tolerant to unreliable networks. Messages may be delayed, duplicated, or reordered, but repeated synchronization attempts eventually bring all nodes to a consistent state.

2.3 Deterministic Merge and Ordering

To ensure that all nodes converge to the same log, the system employs a deterministic merge strategy. When nodes receive updates from peers, they perform a set-union merge of entries, eliminating duplicates and incorporating new data.

However, simple set union is not sufficient to guarantee consistent ordering across nodes. To address this, each entry is assigned a Lamport timestamp, which captures the causal ordering of events. When merging logs, entries are sorted by Lamport timestamp with a deterministic tie-breaker (e.g., message identifier).

This ensures that even when multiple nodes generate updates concurrently, all replicas resolve conflicts in the same way, leading to identical log orderings.

2.4 Control Plane: Membership and Authorization

While the data plane prioritizes availability, the control plane is responsible for enforcing membership and access control policies. This separation allows the system to maintain strong guarantees for authorization without compromising the responsiveness of local operations.

The control plane maintains an Access Control List (ACL) and exposes an authorization interface. Each message includes an epoch value, representing the version of the membership state under which it was generated. When a node receives a message, it queries the control-plane service to verify whether the message's epoch is still valid.

If the epoch is valid, the message is accepted and merged into the log. Otherwise, it is rejected and discarded. By separating policy definition (control plane) from enforcement (data plane), the system achieves both flexibility and correctness.

2.5 Hybrid AP/CP Model

The system follows a hybrid model that combines availability-first (AP) behavior for data operations with consistency-oriented (CP) behavior for membership control.

The data plane operates in an AP mode where local writes always succeed, nodes continue operating under partitions, synchronization happens asynchronously. The control plane operates in a CP mode where membership changes require coordination and authorization decisions are globally consistent.

This hybrid design enables the system to provide strong guarantees where necessary (membership correctness) while maintaining high availability for regular operations. As outlined in the proposal, this separation allows the system to tolerate network failures and still ensure that all nodes eventually converge without violating access constraints .

2.6 Fault Tolerance and Recovery

The system is designed to tolerate crash-stop failures and intermittent connectivity. Nodes can fail and later rejoin the system without requiring global coordination.

When a node recovers, it synchronizes with peers using the same gossip-based mechanism. Since logs are append-only and merges are idempotent, the node can safely replay missed updates without duplicating entries. This replay-based recovery ensures that nodes eventually catch up to the latest state.

Additionally, because synchronization is driven by summaries rather than strict coordination, nodes can recover from partitions and resume normal operation without requiring a centralized leader for data replication.

3. Implementation

3.1 System Components

The system consists of modules for log storage, logical clocks, synchronization (SyncLogs), client-side update generation, and a control-plane authorization service.

3.2 Communication via gRPC

Nodes communicate using gRPC through a single RPC: SyncLogs(SyncLogsRequest) to SyncLogsResponse

Requests include the sender identity, a log summary, and optionally log entries. Responses include a success flag, the receiver's Lamport time, and its summary. This unified interface supports both lightweight summary exchange and full synchronization.

3.3 Anti-Entropy Synchronization

Synchronization follows a two-phase process, 1. **Summary Exchange:** Nodes exchange summaries to detect state differences. 2. **Conditional Log Transfer:** If summaries differ, missing entries are sent; otherwise, no transfer occurs.

This design reduces redundant communication while ensuring eventual convergence, even under delays or repeated synchronization.

3.4 Merge and Idempotence

The merge operation is idempotent and deterministic. Incoming entries are added only if not already present, using message identifiers for deduplication. Entries are then ordered by Lamport timestamp with deterministic tie-breaking. This ensures no duplication under repeated synchronization, and identical ordering across nodes after convergence.

3.5 Epoch-Based Authorization

Authorization is enforced during synchronization via the checkAuth function. Each message is validated by querying the control-plane service with the sender ID and epoch.

Invalid messages are discarded before merging, ensuring that revoked users cannot inject updates, stale messages are filtered, and membership policies are consistently enforced.

3.6 Fault Tolerance and Recovery

The system supports recovery from crash-stop failures through replay-based synchronization. Restarted nodes exchange summaries with peers and retrieve missing entries.

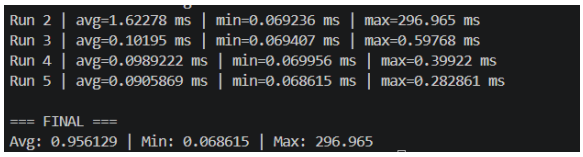
Because merges are idempotent, updates can be safely reapplied without duplication. The append-only log further simplifies recovery by avoiding complex state reconstruction.

3.7 Testing and Validation

The implementation is validated through unit and integration tests:

- Log store tests for merge, ordering, and deduplication
- Lamport clock tests for timestamp correctness
- gRPC tests for end-to-end communication
- Raft and crash tests for control-plane behavior

Manual experiments further confirm convergence, replay safety, and correct rejection of invalid messages.



```
Run 2 | avg=1.62278 ms | min=0.069236 ms | max=296.965 ms
Run 3 | avg=0.10195 ms | min=0.069407 ms | max=0.59768 ms
Run 4 | avg=0.0989222 ms | min=0.069956 ms | max=0.39922 ms
Run 5 | avg=0.0905869 ms | min=0.068615 ms | max=0.282861 ms

=== FINAL ===
Avg: 0.956129 | Min: 0.068615 | Max: 296.965
```

Figure 1: Latency measurements across multiple runs under sequential request workload, showing average, minimum, and maximum latency values, including occasional spikes under higher load.

4. Performance Evaluation

We evaluate our system along the key dimensions outlined in our proposal: convergence, ordering consistency, membership enforcement, and performance. Our experiments are conducted on a local multi-process setup, where each node runs as an independent process and communicates via gRPC.

4.1 Convergence

To evaluate convergence, we deploy three nodes and repeatedly synchronize logs using the gossip-based protocol. Starting from an empty state, one node generates updates while others receive them through the synchronization process.

We observe that after a small number of synchronization rounds, all nodes reach identical states, as verified by matching log sizes and hash values. This confirms that the anti-entropy mechanism ensures eventual consistency, even under asynchronous communication and repeated message exchange.

4.2 Replay and Idempotence

We evaluate the system's behavior under repeated synchronization by invoking the client multiple times with the same set of log entries. The first synchronization transfers the required entries to peers, while subsequent runs perform only summary exchanges.

This demonstrates that the merge operation is idempotent, and that the anti-entropy protocol avoids unnecessary data transfer once convergence is achieved.

4.3 Membership Enforcement

To evaluate membership enforcement, we simulate both valid and invalid message scenarios using epoch-based authorization.

In the valid case, all messages are accepted and merged successfully. In the invalid case, messages with outdated or incorrect epoch values are rejected by the data plane after verification with the control-plane service.

Importantly, rejecting invalid messages does not prevent the system from converging on the remaining valid state. This demonstrates that the

integration of control-plane authorization does not interfere with data-plane consistency.

4.4 Ordering Consistency

We evaluate ordering consistency by introducing concurrent updates across nodes and verifying that all replicas resolve these updates in the same order.

Using Lamport clocks combined with deterministic tie-breaking, all nodes produce identical log orderings after synchronization. This confirms that the system achieves deterministic total ordering, even under concurrent updates.

4.5 Convergence Latency

We measure convergence latency as the time required for nodes to synchronize their logs after updates are introduced. We approximate this by measuring the execution time of the client-triggered synchronization process across multiple nodes.

In our experiments, a single synchronization round across three nodes completed in approximately **100–110 ms,** as measured using system timing utilities. A representative run produced:

Figure 2: Single synchronization run across three nodes, showing end-to-end latency of approximately 100 ms for a client-triggered SyncLogs operation.

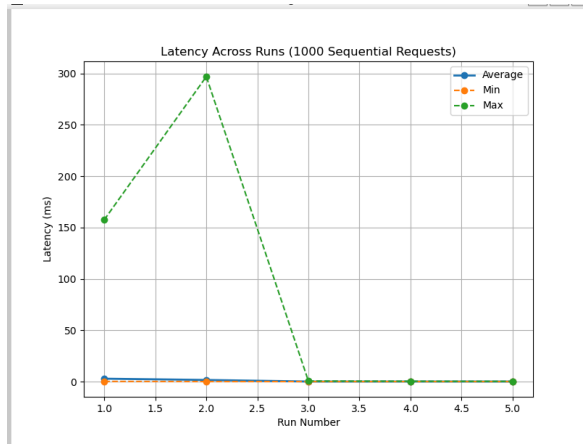


Figure 3: Latency across multiple runs for 1000 sequential requests, showing average, minimum, and maximum latency.

These results indicate that convergence is achieved quickly under local execution conditions. The use of summary-based synchronization reduces unnecessary data transfer, allowing nodes to determine whether updates are needed before sending full log entries. As a result, subsequent synchronization attempts complete significantly faster, as only lightweight summary exchanges are performed.

While these measurements are taken in a single-machine environment and do not reflect network variability, they demonstrate that the system converges efficiently and scales with repeated synchronization attempts.

4.6 Fault Tolerance and Recovery

We evaluate fault tolerance by simulating node restarts and rejoining behavior. After a node is restarted, it reconnects to peers and performs synchronization using the same gossip mechanism without requiring global coordination.

This demonstrates that the system supports recovery through replay and gossip-based synchronization, consistent with the design goals.

5. Limitations and Future Work

While the system achieves availability, convergence, and membership enforcement, it has several limitations. First, evaluation is conducted in a single-machine environment, which does not capture real network conditions such as latency variability or failures at scale. Second, the authorization layer is simplified using a mock service and does not reflect real-world security mechanisms such as cryptographic verification or key management. Finally, the system assumes non-adversarial nodes and does not address Byzantine faults, which would require additional mechanisms such as signatures or quorum validation.

Future work includes deploying the system across multiple machines to evaluate performance under realistic network conditions, integrating stronger security mechanisms such as digital signatures, and improving control-plane coordination using a full consensus protocol. Extending the system to support more general CRDTs and richer data models would further broaden its applicability.

6. Code

You can find our C++ Implementation on:
https://github.com/CSCI546-26sp/final-project-doohickey-engineering